

Федеральное государственное автономное образовательное учреждение
высшего образования «Московский физико-технический институт
(национальный исследовательский университет)

На правах рукописи

Герасимов Илья Александрович

**Исследование и применение многопоточности в системах
игрового искусственного интеллекта**

Специальность 1.3.2 —
«Прикладная математика и информатика»

Бакалаврская дипломная работа

Научный руководитель:
Протасов Антон Андреевич

Долгопрудный — 2026

Аннотация

Работа посвящена применению многопоточности в системах игрового искусственного интеллекта. Рассматривается задача массового обновления ИИ-агентов в условиях ограниченного бюджета кадра. Проанализированы особенности игровой ИИ-нагрузки, риски многопоточного выполнения, task-based подходы, пул потоков, work stealing и влияние размера задачи grainSize.

В практической части разработан прототип системы задач на C++20 с тремя стратегиями выполнения: последовательной, на основе пула потоков и с кражей задач. Помимо независимого обновления агентов реализован режим SnapshotNeighbors, в котором агенты читают неизменяемый снимок состояния соседей и записывают результаты в отдельную фазу. Такой режим моделирует межагентные зависимости без прямой конкурентной записи в общее состояние мира.

Проведены серии бенчмарков при различном числе агентов, потоков, размере задач, доле тяжёлых агентов и радиусе межагентного восприятия. Эксперименты показали, что многопоточность даёт выигрыш только при достаточном объёме полезной работы. Слишком мелкое разбиение создаёт накладные расходы, которые могут превышать пользу от параллелизма; при тяжёлой нагрузке и межагентных чтениях масштабирование выражено лучше, но преимущество work stealing не является универсальным.

По результатам исследования сформулированы рекомендации: подбирать grainSize экспериментально, избегать микрозадач, использовать простой пул потоков для равномерной нагрузки и применять work stealing только при подтверждённом дисбалансе задач.

Оглавление

Введение	5
1 Многопоточность в системах игрового искусственного интеллекта	7
1.1 Игровой ИИ как вычислительная нагрузка	7
1.2 Task-based подход	8
1.3 Thread Pool и Work Stealing	8
1.4 Связь с практическими job systems	9
1.5 Риски многопоточного ИИ	10
1.6 Паттерны корректного обновления агентов	11
1.6.1 Режим межагентных зависимостей на основе снимка состояния	12
1.7 Метрики оценки	14
1.8 Выводы по главе	15
2 Проектирование и реализация системы задач	16
2.1 Постановка задачи	16
2.2 Архитектура	16
2.3 Реализация планировщиков	17
2.4 Гранулярность задач	18
2.5 Модель нагрузки и воспроизводимость	18
2.6 Контроль измерений	19
2.7 Бенчмаркинг	19
2.8 Интерпретация результатов реализации	20
2.9 Выводы по главе	20
3 Экспериментальное исследование	21
3.1 Методика	21
3.2 Лёгкая равномерная нагрузка	21
3.3 Относительное сравнение на лёгкой нагрузке	23
3.4 Влияние grainSize	24
3.5 Тяжёлая неоднородная нагрузка	26

3.6	Стабильность времени кадра	26
3.7	Размер задачи при тяжёлой нагрузке	28
3.8	Масштабирование и сравнение планировщиков	28
3.9	Исследование межагентных зависимостей на основе снимка состояния	31
3.10	Практические рекомендации и ограничения	35
3.11	Выводы по главе	36
Заключение		37

Введение

Современные игровые системы предъявляют жёсткие требования к производительности искусственного интеллекта. ИИ должен обновлять состояние большого числа NPC, реагировать на действия игрока, взаимодействовать с физикой, анимацией и навигацией и при этом укладываться в ограниченный бюджет кадра. При целевой частоте 60 FPS весь кадр занимает около 16,67 мс, а подсистема ИИ получает только часть этого времени. Поэтому даже простая логика отдельного агента становится заметной нагрузкой при масштабировании до тысяч объектов.

Рост числа ядер в процессорах сам по себе не ускоряет игровую логику. Чтобы использовать аппаратный параллелизм, необходимо декомпозировать вычисления, контролировать доступ к общим данным, выбирать подходящую гранулярность задач и учитывать накладные расходы планировщика. Для игрового ИИ это особенно важно: агенты часто читают общее состояние мира, конкурируют за ресурсы, имеют разную стоимость обновления и должны выдавать воспроизводимый результат.

Объектом исследования являются системы игрового ИИ, выполняющие массовое обновление агентов в реальном времени. Предмет исследования — методы организации многопоточного выполнения таких обновлений: task-based parallelism, пул потоков, work stealing и chunk-based разбиение работы.

Цель работы — исследовать условия, при которых многопоточное обновление ИИ-агентов уменьшает время кадра, и определить ситуации, где накладные расходы делают распараллеливание неэффективным. Для достижения цели решались следующие задачи:

- рассмотреть игровую ИИ-нагрузку как объект параллельной обработки;
- проанализировать основные проблемы многопоточности: гонки данных, взаимные блокировки, недетерминизм, накладные расходы и дисбаланс;
- спроектировать экспериментальную систему обновления агентов;
- реализовать последовательную стратегию, пул потоков и work stealing;

- реализовать и исследовать режим межагентных зависимостей на основе снимка состояния мира;
- провести бенчмарки при разных значениях N , p , `grainSize` и `HeavyRatio`;
- сравнить результаты по времени кадра, ускорению, эффективности и пропускной способности.

Практическая значимость работы состоит в разработке прототипа `benchmark framework` и в формулировании рекомендаций по выбору granularity, стратегии планирования и способа организации межагентных чтений. Полученные выводы не претендуют на универсальные численные константы для всех игровых движков, но показывают устойчивые закономерности, которые следует учитывать при проектировании многопоточного ИИ с общим состоянием мира.

Структура работы соответствует поставленным задачам. В первой главе рассматриваются особенности игрового ИИ как вычислительной нагрузки и основные подходы к `task-based` исполнению. Во второй главе описывается разработанная экспериментальная система, методика формирования задач и режим межагентных чтений на основе снимка состояния. В третьей главе приводятся результаты измерений, сравнение планировщиков, анализ `SnapshotNeighbors` и практические рекомендации.

1 Многопоточность в системах игрового искусственного интеллекта

1.1 Игровой ИИ как вычислительная нагрузка

Игровой искусственный интеллект отличается от автономных ИИ-систем тем, что работает внутри фиксированного цикла кадра. Его задача — не найти математически оптимальное действие любой ценой, а быстро сформировать достаточно правдоподобное и согласованное поведение. В одном кадре движок также выполняет физику, анимацию, ввод, звук, сетевую синхронизацию и подготовку рендеринга, поэтому ИИ обычно располагает небольшим и изменяющимся временным бюджетом.

Если целевая частота равна F кадров в секунду, то время кадра равно

$$T_{\text{frame}} = \frac{1000}{F} \text{ мс.} \quad (1.1)$$

Для 60 FPS это 16,67 мс. Важна не только средняя стоимость обновления, но и разброс: отдельный тяжёлый кадр может вызвать заметную просадку даже при приемлемом среднем значении.

Типичная нагрузка игрового ИИ состоит из большого числа однотипных операций над агентами: обновления таймеров, выбора состояния поведения, обработки локального движения, проверки условий, оценки целей и формирования команд. Пусть один агент требует в среднем c микросекунд, а число агентов равно N . Тогда грубая оценка последовательного времени имеет вид

$$T_{\text{seq}} \approx N \cdot c + T_{\text{shared}}, \quad (1.2)$$

где T_{shared} — стоимость общих подсистем и синхронизации. Даже при небольшом c рост N быстро делает последовательное обновление узким местом.

Массовое обновление агентов естественно выражается как `parallel for`: итерация i обрабатывает агента i . Такая модель корректна, если агент читает неизменяемый снимок мира и пишет результат в собственный буфер. Именно эта идея лежит в основе интерфейсов наподобие `Unity IJobParallelFor`, где `runtime` распределяет диапазон индексов между `worker`-потоками [3, 2]. На практике независимость часто нарушается: агенты используют общие списки целей, резервируют позиции, отправляют события группе или

обращаются к навигации. Поэтому параллельная фаза должна быть явно отделена от стадии применения результатов.

1.2 Task-based подход

Создание потока под каждую операцию слишком дорого: требуется выделить стек, зарегистрировать поток у ОС и синхронизировать его завершение. Поэтому игровые системы используют набор долговременных рабочих потоков, которым выдаются короткие задачи. Такой подход соответствует `thread pool` и `job system`; аналогичная мотивация описана в документации Microsoft по пулам потоков [11].

В `thread-based` модели разработчик управляет потоками напрямую. В `task-based` модели он описывает небольшие работы и зависимости, а планировщик решает, где и когда их выполнить. Для игрового кадра эта модель удобнее: кадр состоит из множества задач разного размера, и `runtime` может распределять готовые работы между ядрами. `Unity Job System`, `Unreal Tasks System` и `oneTBB` используют разные реализации этой идеи, но общий принцип остаётся тем же: работа описывается на уровне задач, а потоки являются ресурсом планировщика [1, 5, 9].

Для массового ИИ важна гранулярность. Одна задача на агента даёт слишком много служебной работы, а одна задача на весь массив не использует параллелизм. Поэтому применяется `chunk-based parallelFor`: массив агентов делится на блоки размера g , и число задач приблизительно равно

$$M = \left\lceil \frac{N}{g} \right\rceil. \quad (1.3)$$

Малый g улучшает балансировку, но увеличивает расходы на постановку задач, синхронизацию и работу очередей. Большой g уменьшает накладных расходов, но может оставить часть рабочих потоков без работы.

1.3 Thread Pool и Work Stealing

Пул потоков является наиболее простой практической схемой выполнения задач. На этапе инициализации создаётся фиксированное число рабочих потоков, после чего каждый кадр помещает задачи в общую очередь. Потоки извлекают готовые задачи, выполняют их и переходят к следующей работе. Достоинство такого подхода — простота реализации и низкая стоимость

поддержки при умеренном числе задач. Недостаток — общая очередь может стать точкой конкуренции: чем больше потоков одновременно обращаются к ней, тем выше стоимость синхронизации.

Для игрового ИИ пул потоков удобен, когда задачи имеют близкую стоимость. Например, если все агенты выполняют одинаковое лёгкое обновление, равномерное разбиение массива на блоки обычно достаточно хорошо распределяет работу между потоками. В этом случае усложнение планировщика может не дать выигрыша, потому что основное время тратится не на простой рабочих потоков, а на полезное выполнение или на служебную работу очереди.

Work stealing строится иначе. Каждый worker имеет локальную очередь задач. Обычно поток берёт работу из своей очереди, а если она пуста, пытается украсть задачу у другого worker-а. Такая схема уменьшает простой при неоднородной нагрузке: если один поток получил несколько дорогих задач, другие могут забрать часть оставшейся работы. Этот подход используется в task scheduler-ах, где нагрузка заранее неизвестна или задачи порождают новые задачи во время выполнения [9].

Однако work stealing не является бесплатной оптимизацией. Требуются дополнительные очереди, атомарные операции, попытки steal и политика выбора жертвы. Если задачи однородны или слишком мелки, расходы на более сложное управление могут перекрыть выигрыш от балансировки. Поэтому сравнение ThreadPool и WorkStealing в работе выполняется экспериментально: теоретически можно ожидать преимуществ work stealing при дисбалансе, но реальный результат зависит от стоимости задач, размера блока и реализации очередей.

1.4 Связь с практическими job systems

Современные игровые движки используют job systems не только для ИИ, но и для анимации, физики, подготовки данных рендеринга, обработки частиц и фоновой загрузки ресурсов. Общая причина одна: кадр состоит из набора работ, часть которых можно выполнять параллельно, но зависимости между ними должны быть явно учтены. Поэтому многие движки переходят от ручного управления потоками к графам задач, где система знает, какие работы уже готовы, а какие должны ждать завершения предыдущих стадий.

Unity Job System предоставляет разработчику модель задач с ограниче-

ниями на доступ к данным и поддержкой `parallel for`-работ [1, 2]. Это важно именно для ИИ: система безопасности помогает обнаруживать конфликтующие обращения к данным, а `IJobParallelFor` естественно описывает обновление множества сущностей. Unreal Engine использует собственные системы задач и инструменты анализа `task graph`, позволяющие распределять работу между потоками и искать узкие места в кадре [5, 6].

В докладах разработчиков игровых движков подчёркивается, что переход к `task-based multithreading` требует не только библиотеки задач, но и изменения мышления разработчика: подсистемы должны формировать независимые работы, минимизировать синхронные ожидания и передавать результаты через контролируемые фазы [7, 8]. Это напрямую связано с игровой ИИ-логикой. Если агент в середине задачи обращается к произвольным системам мира, планировщик не может эффективно распределять работу и гарантировать корректность.

Разрабатываемая в работе система является упрощённой моделью таких `job systems`. Она не реализует полноценный граф зависимостей, но сохраняет ключевой элемент: логика обновления агентов отделена от политики исполнения. Благодаря этому можно исследовать один из центральных параметров реальных систем — размер задач. Даже если `production job system` сложнее, проблема гранулярности остаётся: слишком мелкие задачи порождают накладных расходов, а слишком крупные ухудшают балансировку.

1.5 Риски многопоточного ИИ

Главная проблема параллельного ИИ — общее изменяемое состояние. Если несколько потоков одновременно читают и изменяют одну структуру без согласованной дисциплины доступа, результат зависит от порядка выполнения. В игре это может проявиться не как немедленное падение, а как редкое неверное решение NPC, конфликт за укрытие или нестабильная анимация. `Safety system Unity` прямо связывает гонки данных с недетерминированным поведением `job`-системы [4].

`Mutex`, `atomic` операции и условные переменные решают часть задач синхронизации, но не являются бесплатными. Частые блокировки превращают параллельную программу в очередь ожиданий, а некорректный порядок захвата ресурсов может привести к взаимной блокировки. В игровых системах предпочтительно проектировать данные так, чтобы параллельная фаза

работала с только для чтения входами и локальными выходными буферами, а конфликтующие изменения применялись после завершения задач.

Другой источник потерь — накладные расходы планировщика. Они включают создание объектов задач, работу очередей, пробуждение потоков, атомарные счётчики завершения и ожидание барьеров. Если полезная работа в задаче мала, эти расходы становятся больше самой ИИ-логики. Поэтому фраза «добавить многопоточность» не означает автоматического ускорения.

Дисбаланс нагрузки возникает, когда разные задачи имеют разную стоимость. В ИИ это типичная ситуация: один агент простаивает, другой выполняет сенсорику, третий иницирует поиск пути. Work stealing пытается уменьшить простой потоков: каждый worker имеет локальную очередь и может забрать работу у другого worker-а, если его очередь пуста. Такой механизм полезен при неоднородной нагрузке, но сам stealing также требует синхронизации и не всегда окупается.

Отдельно стоит учитывать недетерминизм. Даже при отсутствии гонок порядок завершения задач может меняться от запуска к запуску. Если результаты применяются сразу в общие структуры, разные расписания потоков приведут к разным игровым состояниям. Для одиночной игры это усложняет отладку, а для сетевой игры может нарушить синхронизацию клиентов. Поэтому практические job systems часто используют фазовую модель: сначала задачи считают локальные результаты, затем main thread или отдельная стадия commit применяет их в фиксированном порядке.

Наконец, параллелизм может ухудшить кэш-локальность. Если состояние агента хранится как большой объект с редко используемыми полями, рабочие потоки загружают лишние данные и конкурируют за пропускную способность памяти. Для массовых проходов выгоднее плотные массивы данных и предсказуемый порядок обхода. В данной работе эта сторона намеренно упрощена: цель экспериментов — оценить планирование задач и гранулярность, а не сравнить разные layout-ы данных.

1.6 Паттерны корректного обновления агентов

На практике многопоточный ИИ редко строится как произвольный параллельный доступ к объектам сцены. Более устойчивый подход — разделение кадра на фазы с разными правилами доступа. В начале кадра формируется неизменяемое состояние, которое могут читать все задачи: позиции, флаги

видимости, навигационные данные, параметры команд и результаты предыдущего кадра. Затем рабочие потоки выполняют локальные вычисления и записывают выходы в отдельные буферы. В конце кадра результаты применяются в фиксированном порядке.

Такой подход уменьшает число гонок и делает поведение ближе к последовательной модели. Например, если несколько агентов выбирают укрытия, параллельная фаза может только сформировать заявки, а стадия `commit` разрешит конфликт по детерминированному правилу: приоритету агента, расстоянию или стабильному идентификатору. Это дороже, чем немедленная запись в общий объект, но существенно упрощает воспроизводимость и отладку.

Другой распространённый приём — двойная буферизация состояния. Задачи читают состояние кадра k и записывают результаты в буфер кадра $k + 1$. Пока все задачи не завершены, старое состояние не изменяется. После барьера буферы меняются местами. Для ИИ это удобно, когда агенты принимают решения на основе общего снимка мира. Недостаток состоит в дополнительной памяти и возможной задержке на один кадр, но для многих игровых решений такая задержка приемлема.

Третий приём — ограничение параллельной фазы только независимой частью алгоритма. Например, сенсорика и оценка `utility`-функций могут выполняться параллельно, а операции с глобальными ресурсами остаются последовательными или выполняются через отдельные очереди команд. Этот компромисс часто лучше полной синхронизации: он сохраняет основную выгоду от параллелизма и не превращает каждую запись в `mutex`.

В данной работе экспериментальная модель использует именно упрощённый вариант такого подхода: задача обновляет диапазон агентов и не модифицирует глобальные игровые ресурсы. Это позволяет сосредоточиться на вопросе планирования задач. В реальном движке к этому добавились бы зависимости между фазами, граф задач, приоритеты и политика безопасного применения результатов.

1.6.1 Режим межагентных зависимостей на основе снимка состояния

Независимое обновление агентов является удобной базовой моделью, но в реальном игровом ИИ агент редко принимает решение полностью изолированно. Ему может потребоваться информация о соседях: расстояние до ближай-

ших NPC, направление движения группы, плотность толпы, занятые укрытия или локальные угрозы. Если каждый агент во время параллельного update-a будет напрямую читать и изменять актуальное состояние других агентов, порядок выполнения задач начнёт влиять на результат. Один поток может прочитать позицию соседа до изменения, другой — после, а третий одновременно попытается записать новое решение. Такая схема создаёт race conditions и делает поведение зависимым от расписания потоков.

Прямое исправление через mutex-и обычно плохо подходит для массового ИИ. Блокировка вокруг каждого чтения или записи превращает параллельный проход в набор коротких критических секций, а значит, увеличивает ожидания и разрушает предсказуемость времени кадра. Кроме того, при большом числе агентов трудно гарантировать единый порядок захвата ресурсов: два агента могут одновременно пытаться изменить связанные объекты, что повышает риск взаимной блокировки или редких зависаний. Поэтому более практичная дисциплина состоит в том, чтобы не позволять задачам изменять данные, которые другие задачи используют как вход.

Подход на основе снимков состояния разделяет кадр на две смысловые части. В начале кадра формируется только для чтения snapshot состояния мира: позиции, скорости, идентификаторы, параметры видимости и другие поля, необходимые для принятия решений. Все рабочие потоки читают один и тот же снимок, поэтому для них состояние соседей не меняется в середине вычисления. Агент может просмотреть элементы в пределах радиуса восприятия, оценить локальную плотность или выбрать направление обхода, но эти операции остаются чтением неизменяемых данных.

Результаты параллельной фазы записываются не в общий объект агента, а в отдельный выходной буфер: новую скорость, намерение, заявку на ресурс или команду движения. После завершения всех задач выполняется фаза применения изменений. В ней результаты применяются к основному состоянию мира в фиксированном порядке. Если несколько агентов претендуют на один ресурс, конфликт разрешается детерминированным правилом: например, по приоритету, расстоянию или стабильному идентификатору. Таким образом, параллельная часть остаётся преимущественно только для чтения, а потенциально конфликтующие записи концентрируются в контролируемой стадии.

Преимущество такого режима состоит в том, что он сохраняет есте-

ственную модель поведения толпы и при этом не требует синхронизации на каждой операции чтения. Для benchmark-а это важно: можно исследовать не только накладные расходы планировщика, но и стоимость регулярного чтения соседних агентов из памяти. Рост радиуса восприятия увеличивает объём данных, которые задача должна просмотреть, и тем самым меняет баланс между полезной работой, пропускной способностью памяти и стоимостью планирования.

Ограничение подхода на основе снимков состояния заключается в задержке реакции на изменения внутри кадра. Агент принимает решение по состоянию на начало кадра и не видит результаты, которые соседи вычисляют параллельно. Для многих ИИ-задач такая задержка приемлема, потому что игровой кадр короток, а поведение толпы не требует мгновенной взаимной синхронизации. Однако для жёстких физических контактов, сетевой авторитетной симуляции или ресурсов с немедленным эксклюзивным владением может потребоваться более строгий граф зависимостей и отдельные последовательные участки.

В данной работе режим `SnapshotNeighbors` используется как компромисс между полностью независимым обновлением и произвольным общее изменяемое состояние. Он добавляет межагентные чтения и локальные зависимости, но сохраняет проверяемую структуру выполнения: общий снимок только читается, результаты пишутся в собственные буферы, а применение изменений отделено от параллельной фазы.

1.7 Метрики оценки

Основная метрика для ИИ в реальном времени — среднее время update-кадра $\bar{T}$. Дополнительно важны минимум и максимум, так как они показывают разброс и худшие кадры. Для сравнения с последовательным вариантом используется ускорение

$$S_p = \frac{T_1}{T_p}, \quad (1.4)$$

где T_1 — время последовательного выполнения, а T_p — время при p потоках. Эффективность равна

$$E_p = \frac{S_p}{p}. \quad (1.5)$$

Пропускная способность задаётся как

$$Q = \frac{N}{\overline{T}}. \quad (1.6)$$

В реальных системах рост ускорения ограничен последовательной частью алгоритма и накладными расходами; это соответствует закону Амдала [12]. Поэтому экспериментальная оценка должна учитывать не только число потоков, но и размер задач, стоимость агента и распределение нагрузки.

1.8 Выводы по главе

Массовое обновление ИИ-агентов хорошо подходит для параллелизма по данным только при корректной организации доступа к состоянию мира. Модель, основанная на задачах удобна для игрового кадра, но её эффективность определяется гранулярностью задач и отношением полезной работы к накладным расходам. Для дальнейшего исследования требуется экспериментальная система, позволяющая независимо менять число агентов, число потоков, размер блока и неоднородность нагрузки.

2 Проектирование и реализация системы задач

2.1 Постановка задачи

Практическая часть работы представляет собой `benchmark framework`, а не полноценный игровой движок. Такой выбор позволяет контролируемо менять параметры нагрузки и сравнивать планировщики в одинаковых условиях. Цель реализации — проверить, как `grainSize`, число потоков и неоднородность стоимости агентов влияют на время `update`-фазы.

Модель мира содержит массив агентов. Каждый агент имеет текущую позицию, целевую позицию и процедуру обновления. В обычном режиме агент перемещается к цели и при достижении выбирает новую. Для моделирования неоднородной стоимости вводятся `heavy agents`: доля таких агентов задаётся параметром `HeavyRatio`, а дополнительная стоимость — параметром `HeavyIterations`. Эта модель не имитирует конкретный алгоритм, например `pathfinding`, но даёт управляемую вычислительную нагрузку.

В системе сравниваются три стратегии:

- `SequentialScheduler` — последовательное выполнение всех задач в одном потоке;
- `ThreadPoolScheduler` — выполнение задач фиксированным пулом рабочих потоков через общую очередь;
- `WorkStealingScheduler` — выполнение с локальными очередями рабочих потоков и кражей задач при простое.

2.2 Архитектура

Архитектура построена вокруг интерфейса `IScheduler`. Игровая логика не знает, какой планировщик выполняет задачи: она формирует набор диапазонов агентов и передаёт их в `parallelFor`. Это позволяет запускать одну и ту же симуляцию с разными стратегиями выполнения и корректно сравнивать результаты.

Один кадр проходит одинаковой схеме обработки. `Simulation` определяет диапазон агентов, делит его на блоки размера `grainSize`, создаёт задачи обновления и передаёт их выбранному планировщику. После завершения всех задач фиксируется время кадра. В такой схеме различается только

механизм исполнения: последовательный цикл, общая очередь пула потоков или work-stealing очереди.

Таблица 2.1: Основные компоненты экспериментальной системы

Компонент	Назначение
Agent	Хранит состояние агента и выполняет один шаг обновления.
World	Содержит массив агентов и параметры генерации нагрузки.
Simulation	Организует update-кадр, делит массив на задачи и измеряет время.
IScheduler	Единый интерфейс для последовательного, пул потоков и work-stealing исполнения.
BenchmarkRunner	Запускает серии экспериментов и собирает агрегированные метрики.
CSV Exporter	Сохраняет параметры запуска и результаты в CSV для последующего анализа.

2.3 Реализация планировщиков

`SequentialScheduler` используется как базовая линия. Он получает тот же набор задач, что и параллельные планировщики, но выполняет их в текущем потоке. Это важно для корректного сравнения: последовательный вариант измеряет стоимость самой логики обновления и минимальные расходы на обход диапазонов, но не содержит синхронизации рабочих потоков.

`ThreadPoolScheduler` создаёт набор рабочих потоков и общую очередь задач. При запуске кадра задачи помещаются в очередь, после чего потоки извлекают их до полного завершения набора. Для ожидания конца кадра используется счётчик незавершённых задач и механизм уведомления. Такая схема проста, но при большом числе мелких задач все рабочие потоки часто обращаются к одним и тем же синхронизированным структурам. Поэтому именно в этой реализации хорошо виден рост накладных расходов при малом `grainSize`.

`WorkStealingScheduler` использует локальные очереди рабочих потоков. Новые задачи распределяются между очередями, а поток сначала пытается выполнить локальную работу. Если локальная очередь пуста, он выбирает другую очередь и пытается украсть задачу. Ожидаемый выигрыш возникает тогда, когда часть worker-ов завершает свои задачи раньше других. В этом случае stealing позволяет использовать освободившиеся ядра вместо пассивного ожидания барьера кадра.

Реализация work stealing в данной работе исследовательская. Она доста-

точна для сравнения поведения двух политик распределения задач, но не включает все оптимизации промышленные системы управления задачами: affinity к данным, приоритеты, граф зависимостей, fiber-based ожидание и тонкую настройку backoff-политики. Это ограничение учитывается при интерпретации результатов: работа сравнивает конкретные реализации, а не доказывает абсолютное превосходство или слабость всего класса work-stealing планировщиков.

2.4 Гранулярность задач

Параметр `grainSize` является центральным для эксперимента. Если N агентов делятся на задачи по g агентов, то малый g создаёт много задач и повышает накладные расходы. Большой g уменьшает число задач, но может ухудшить балансировку, особенно если heavy agents распределены неравномерно.

В реализации `parallelFor` каждая задача обновляет непрерывный диапазон индексов. Это упрощает работу с памятью, делает выполнение предсказуемым и соответствует типичному data-parallel проходу. Для последовательного планировщика `grainSize` почти не влияет на алгоритмическую суть, но сохраняется в параметрах, чтобы сравнение с параллельными вариантами выполнялось на одинаковой декомпозиции.

2.5 Модель нагрузки и воспроизводимость

Лёгкая нагрузка соответствует сценарию, где каждый агент выполняет одинаковую недорогую логику. Такой режим полезен для оценки чистых накладных расходов планировщика: если полезная работа мала, становится видно, при каком размере задач многопоточность перестает окупаться.

Тяжёлая нагрузка создаётся добавлением heavy agents. Они выполняют дополнительный вычислительный цикл, число итераций которого задаётся параметром `HeavyIterations`. Доля таких агентов задаётся `HeavyRatio`. Эта модель не привязана к конкретной игровой механике, но похожа на реальные ситуации, где часть NPC выполняет более дорогие операции: поиск пути, сенсорику, оценку целей или групповую координацию.

Для воспроизводимости все серии запускаются по фиксированной сетке параметров. В CSV сохраняются не только итоговые времена, но и

конфигурация запуска: имя планировщика, число агентов, число потоков, `grainSize`, доля `heavy agents` и число итераций тяжелой ветки. Такой формат позволяет пересчитывать ускорение, эффективность и относительные сравнения без ручного переноса данных.

2.6 Контроль измерений

Измерение многопоточной программы чувствительно к случайным факторам: состоянию ОС, фоновой нагрузке, прогреву кэшей, миграции потоков и турборежимам процессора. Поэтому отдельный запуск не должен рассматриваться как достаточное основание для вывода. В работе используются серии измерений, а в CSV сохраняются среднее, минимальное и максимальное время кадра. Среднее значение показывает типичную стоимость `update`-фазы, минимум помогает увидеть нижнюю границу, а максимум фиксирует худшие кадры.

При сравнении планировщиков важно использовать одинаковую конфигурацию. Если сравнивать `ThreadPool` при одном `grainSize` с `WorkStealing` при другом, результат будет отражать не только различие планировщиков, но и различие декомпозиции. Поэтому в таблицах парного сравнения оба планировщика берутся при одинаковых N , p , `grainSize`, `HeavyRatio` и `HeavyIterations`. Такой подход позволяет отделить эффект стратегии планирования от эффекта размера задачи.

Дополнительно учитывается, что ускорение относительно `Sequential` корректен только там, где есть сопоставимый последовательный базовый последовательный вариант. В `light`-серии он присутствует, поэтому можно вычислять S_p и E_p . В `heavy`-серии последовательный перебор всей сетки параметров слишком дорог, поэтому основным показателем становится $S_{WS/TP}$, то есть относительное сравнение двух параллельных реализаций при одинаковой нагрузке.

2.7 Бенчмаркинг

`Benchmark framework` запускает серии измерений с фиксированными параметрами: числом агентов N , числом потоков p , размером задачи `grainSize`, долей `heavy agents` и числом `heavy`-итераций. Для каждого запуска сохраняются `AvgFrameMs`, `MinFrameMs`, `MaxFrameMs`, пропускная способность и параметры конфигурации.

Использование CSV делает обработку результатов воспроизводимой: графики и таблицы строятся не вручную, а из одних и тех же исходных данных. Это важно для сравнения планировщиков, потому что небольшие изменения нагрузки могут менять относительный результат `ThreadPool` и `WorkStealing`.

2.8 Интерпретация результатов реализации

Результаты экспериментов следует читать как оценку конкретной архитектуры и конкретных реализаций планировщиков. Это не уменьшает ценность работы, но задаёт границы выводов. Если заменить структуру данных, алгоритм агента, механизм очередей или аппаратную платформу, абсолютные значения времени изменятся. Поэтому основное внимание уделяется не миллисекундам как универсальным константам, а зависимостям: как меняется результат при уменьшении `grainSize`, росте числа потоков и появлении тяжёлых агентов.

Такой подход близок к инженерной практике оптимизации игровых систем. Сначала строится контролируемый `benchmark`, затем выявляются чувствительные параметры, после чего уже на реальной подсистеме проверяется, сохраняются ли те же закономерности. В данной работе `benchmark` показывает, какие режимы заведомо рискованны: большое число микрозадач при лёгкой логике, слепой выбор `work stealing` без измерений и оценка только по среднему времени кадра.

2.9 Выводы по главе

Разработанная система отделяет модель агента от механизма исполнения и позволяет сравнивать разные планировщики при одинаковой логике обновления. Основным экспериментальным параметром — `grainSize`; он задаёт компромисс между накладными расходами и балансировкой. Такая архитектура достаточна для проверки ключевой гипотезы работы: многопоточность эффективна не сама по себе, а только при подходящем размере задач и характере нагрузки.

3 Экспериментальное исследование

3.1 Методика

Эксперименты проводились на четырёх наборах данных. Базовая серия применялась для первичной проверки всех режимов, light-серия изолировала накладные расходы при равномерной лёгкой нагрузке, heavy-серия моделировала неоднородную стоимость агентов, а interaction_snapshot исследовала межагентные чтения на основе снимка состояния.

Таблица 3.1: Используемые наборы данных

Серия	Строк	Основные параметры
base	1350	$N = 1000 \dots 50000$, $p \in \{1, 2, 4, 8\}$, grainSize $\in \{16, 64, 256, 1024, 2048\}$, разные HeavyRatio.
light	486	$N = 10000 \dots 1000000$, $p \in \{1, 2, 4, 8\}$, grainSize от 1 до 65536, HeavyRatio=0.
heavy	384	$N = 10000 \dots 200000$, $p \in \{4, 8\}$, grainSize $\in \{64, 256, 1024, 4096\}$, HeavyRatio $\in \{0, 01, 0, 10, 0, 25\}$.
interaction snapshot	135	$N \in \{10000, 50000, 100000\}$, $p \in \{1, 4, 8\}$, grainSize $\in \{64, 256, 1024\}$, PerceptionRadius $\in \{2, 5, 10\}$.

Основная метрика анализа — среднее время update-кадра. Для light-серии и режима interaction_snapshot также вычислялось ускорение относительно Sequential. Для heavy-серии последовательный базовый последовательный вариант не запускался на всей сетке, поэтому ThreadPool и WorkStealing сравнивались попарно:

$$S_{WS/TP} = \frac{T_{ThreadPool}}{T_{WorkStealing}}. \quad (3.1)$$

Если $S_{WS/TP} > 1$, быстрее WorkStealing; если значение меньше единицы, быстрее ThreadPool.

3.2 Лёгкая равномерная нагрузка

В light-серии HeavyRatio=0, поэтому все агенты имеют близкую стоимость обновления. На рис. 3.1 показано среднее время кадра при $p = 8$ и grainSize=1024.

При достаточно крупном размере задачи оба параллельных планировщика дают ускорение относительно Sequential. Однако результат зависит от

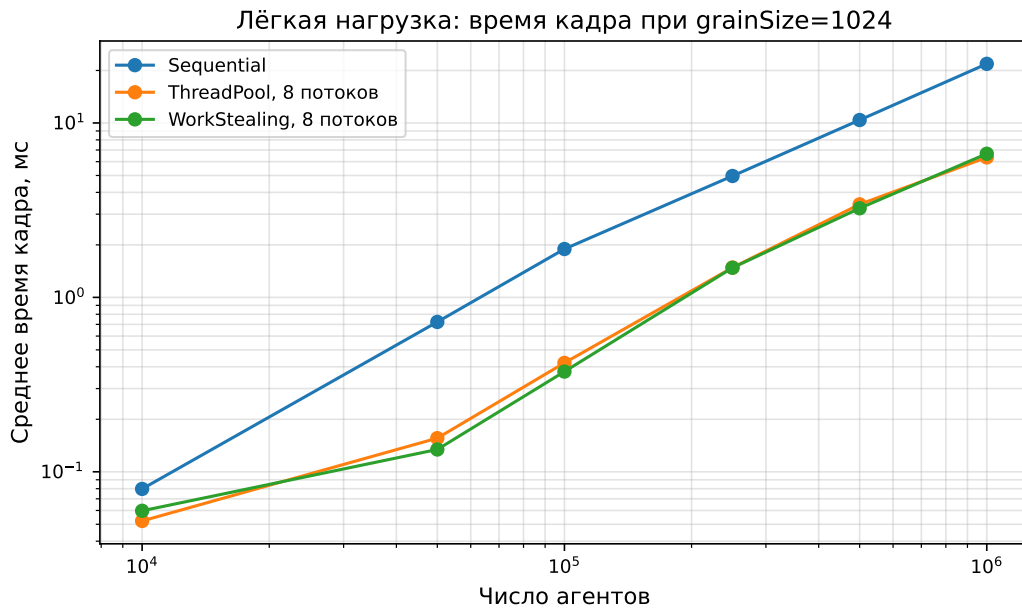


Рис. 3.1: Среднее время update-кадра на лёгкой нагрузке при grainSize=1024 и $p = 8$.

размера входа: максимум ускорения достигается не всегда на самом большем N . Это указывает на влияние расходов очередей, синхронизации и кэш-эффектов.

Ускорение для того же среза показано на рис. 3.2. Значения выше единицы означают выигрыш относительно последовательного обновления. В области $N = 50000 \dots 100000$ ускорение достигает максимума, после чего не растёт монотонно. Это согласуется с тем, что рост числа агентов увеличивает полезную работу, но одновременно меняет поведение памяти, очередей и распределения задач.

Показательный срез приведён в табл. 3.2. При $N = 50000$ и $N = 100000$ лучшим оказывается WorkStealing, но при $N = 10000$, $N = 500000$ и $N = 1000000$ минимальное время даёт ThreadPool. Следовательно, в равномерной лёгкой нагрузке нельзя заранее выбрать универсально лучший планировщик.

Таблица 3.2: Лучшие парные конфигурации light-серии

N	p	grainSize	Seq, мс	TP, мс	WS, мс	Лучший
10000	8	1024	0,080	0,052	0,060	TP
50000	8	1024	0,723	0,156	0,134	WS
100000	8	1024	1,894	0,421	0,375	WS
250000	8	4096	4,951	1,458	1,445	WS
500000	8	4096	11,265	3,124	3,311	TP
1000000	8	1024	21,827	6,348	6,661	TP

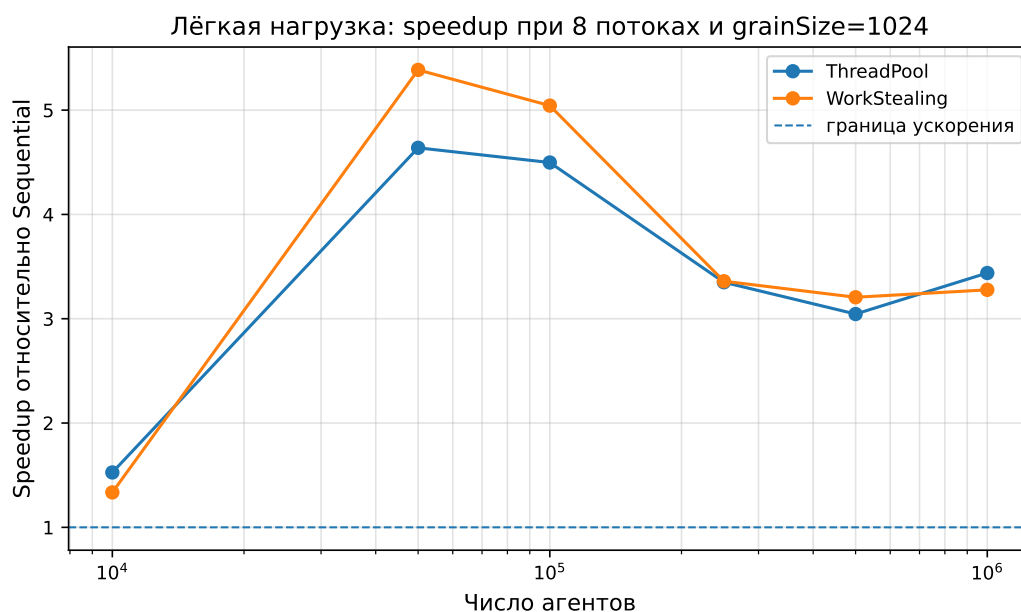


Рис. 3.2: Ускорение относительно Sequential на лёгкой нагрузке при $p = 8$ и $\text{grainSize}=1024$.

3.3 Относительное сравнение на лёгкой нагрузке

Отдельный вопрос состоит в том, какой из двух параллельных планировщиков быстрее при одинаковом grainSize . На лёгкой нагрузке это сравнение не совпадает с вопросом об ускорении относительно Sequential. Возможна ситуация, где WorkStealing быстрее ThreadPool, но оба варианта проигрывают последовательному выполнению из-за слишком мелких задач.

На рис. 3.3 показано отношение времени ThreadPool к времени WorkStealing при разных grainSize . Значения выше единицы означают, что WorkStealing быстрее. В полной light-серии он быстрее в 149 из 216 парных конфигураций. Особенно часто это происходит при малом grainSize , где локальные очереди могут снижать конкуренцию за общую очередь. Однако практический вывод по таким конфигурациям остаётся отрицательным, если они не дают ускорения относительно Sequential.

Таким образом, сравнение планировщиков следует выполнять в два этапа. Сначала нужно проверить, окупается ли само распараллеливание относительно последовательного обновления. Затем среди конфигураций, где ускорение действительно есть, можно выбирать более быстрый параллельный планировщик. Такой порядок защищает от ложного вывода, когда один параллельный вариант лучше другого, но оба непригодны для реального кадра.

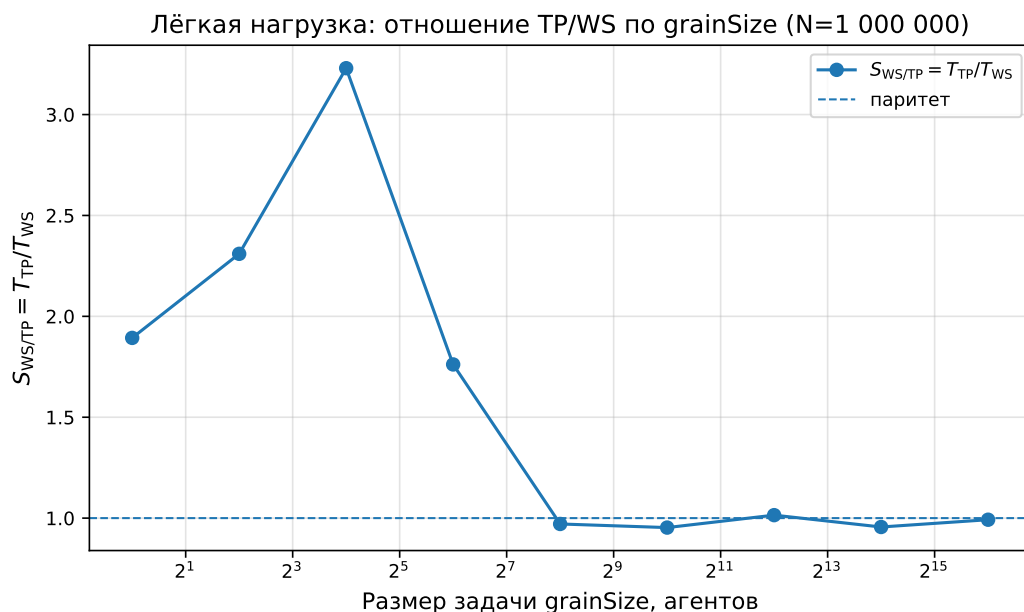


Рис. 3.3: Относительное сравнение WorkStealing и ThreadPool на лёгкой нагрузке.

3.4 Влияние grainSize

Размер задачи оказывает наибольшее влияние при лёгкой нагрузке. При $\text{grainSize}=1$ на $N = 1000000$ создаётся миллион задач, и расходы планирования доминируют над полезной работой. На рис. 3.4 показано, что увеличение grainSize резко снижает время кадра до области плато.

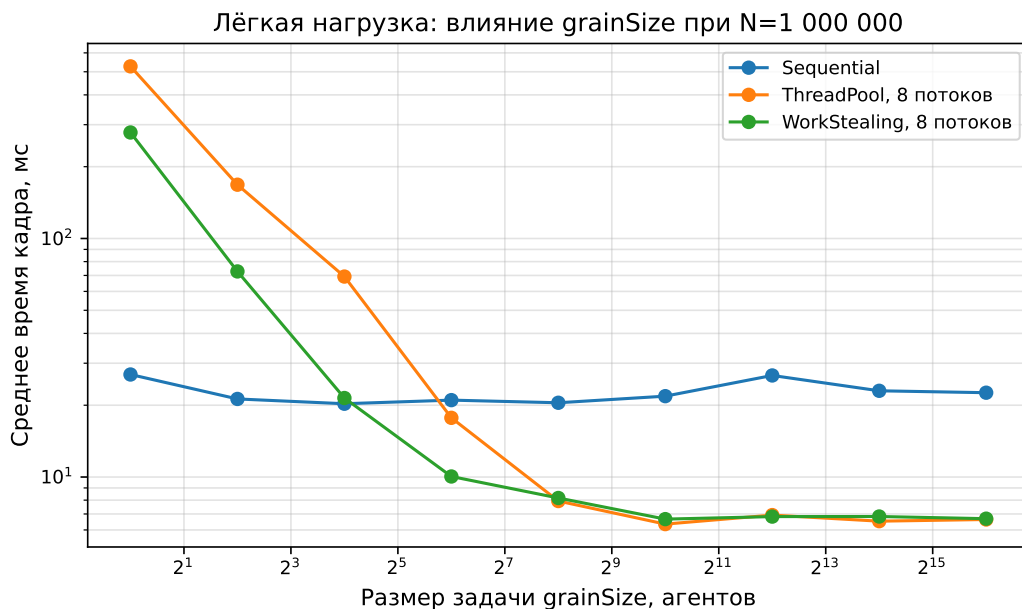


Рис. 3.4: Влияние grainSize на среднее время кадра при лёгкой нагрузке, $N = 1000000$, $p = 8$.

При размере задачи 1 время ThreadPool равно 526,732 мс, а WorkStealing — 278,228 мс. При размере 1024 эти значения уменьшаются

до 6,348 и 6,661 мс соответственно. Значит, изменение только `grainSize` уменьшает время `ThreadPool` примерно в 83 раза, а `WorkStealing` — примерно в 42 раза.

Относительный накладных расходов показан на рис. 3.5 и в табл. 3.3. При размере задачи 1 параллельные варианты значительно медленнее последовательного: `ThreadPool` проигрывает в 19,58 раза, а `WorkStealing` — в 10,34 раза. При размере 16 `WorkStealing` находится около границы окупаемости, но `ThreadPool` всё ещё заметно медленнее. Начиная с `grainSize=64` оба планировщика переходят в область полезного ускорения.

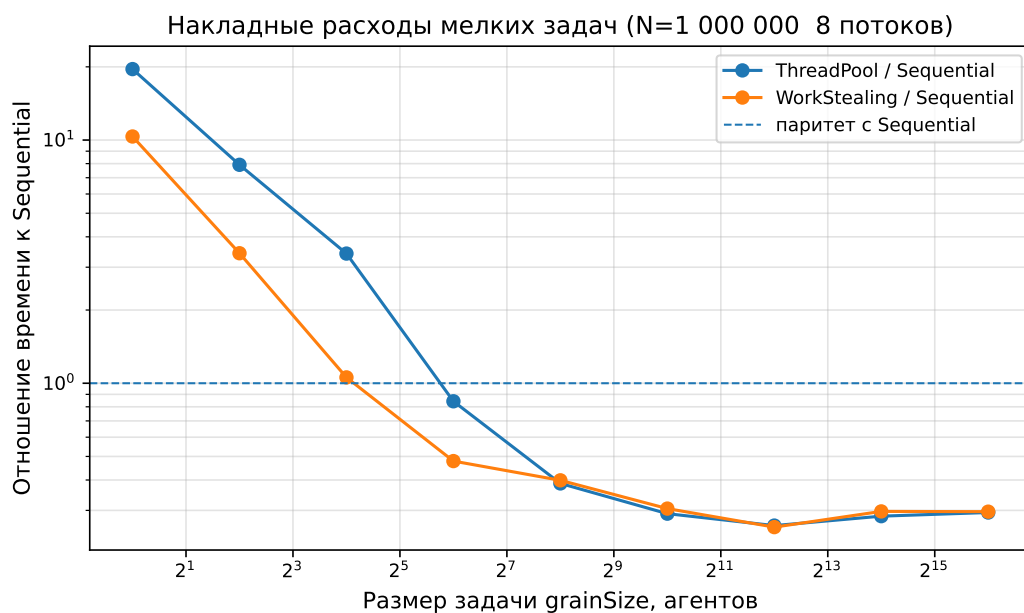


Рис. 3.5: Отношение времени параллельного выполнения к времени `Sequential` при $N = 1000000$, $p = 8$.

Таблица 3.3: Накладные расходы при $N = 1000000$, $p = 8$

grainSize	Seq, мс	TP, мс	WS, мс	TP/Seq	WS/Seq
1	26,903	526,732	278,228	19,58	10,34
4	21,248	168,066	72,761	7,91	3,42
16	20,288	69,247	21,440	3,41	1,06
64	21,008	17,708	10,052	0,84	0,48
1024	21,827	6,348	6,661	0,29	0,31
4096	26,655	6,925	6,825	0,26	0,26

Инженерный вывод состоит в том, что для лёгких агентов микрозадачи непрактичны. Начиная примерно с `grainSize=64`, накладные расходы перестают полностью перекрывать пользу от параллелизма, но дальнейшее увеличение размера задачи не гарантирует монотонного улучшения.

3.5 Тяжёлая неоднородная нагрузка

В heavy-серии часть агентов выполняет дополнительную вычислительную ветку. На рис. 3.6 показано, как среднее время кадра растёт с увеличением HeavyRatio при $N = 200000$, $p = 8$, $\text{grainSize}=1024$ и $\text{HeavyIterations}=10000$.

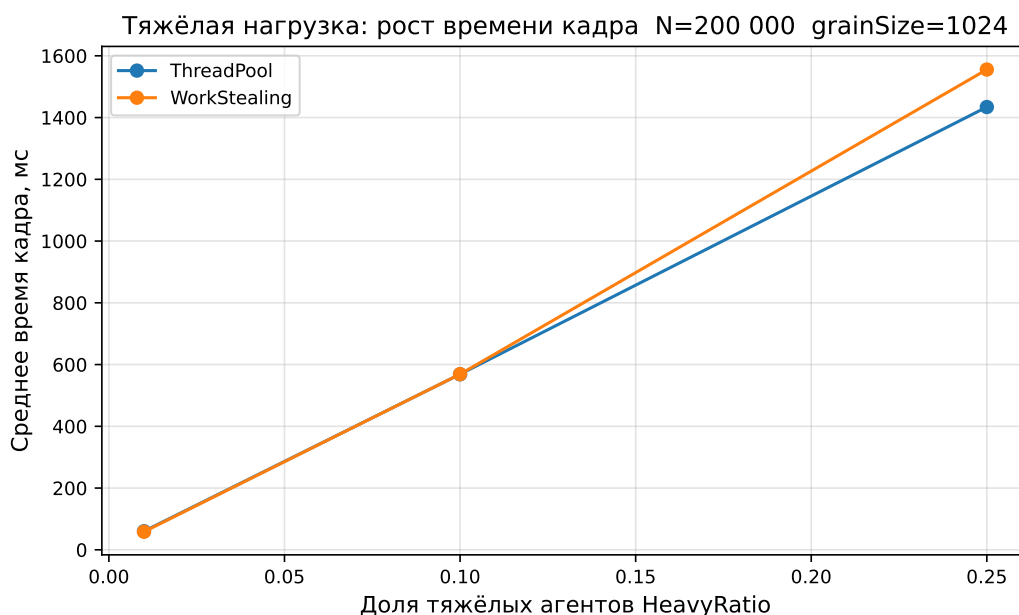


Рис. 3.6: Рост среднего времени кадра при увеличении доли тяжёлых агентов.

Для ThreadPool время возрастает с 60,240 мс при HeavyRatio=0,01 до 568,645 мс при HeavyRatio=0,10 и 1434,113 мс при HeavyRatio=0,25. Для WorkStealing соответствующие значения равны 58,530, 568,864 и 1555,543 мс. Рост близок к росту объёма тяжёлой работы, что подтверждает пригодность модели heavy agents для управляемого создания нагрузки.

В лучших конфигурациях heavy-серии преимущество WorkStealing невелико и неустойчиво (табл. 3.4). При доле тяжёлых агентов 0,25 и 10000 итерациях лучший результат показывает ThreadPool. Это означает, что кража задач не гарантирует выигрыш даже при неоднородной нагрузке: её стоимость должна быть меньше стоимости простоя потоков.

3.6 Стабильность времени кадра

Среднее время кадра не всегда достаточно для оценки игровой системы. Если среднее значение приемлемо, но отдельные кадры имеют большой максимум, игрок всё равно увидит рывки. Поэтому в heavy-серии дополнительно рассматривались MinFrameMs и MaxFrameMs.

Таблица 3.4: Лучшие парные конфигурации heavy-серии при $N = 200000$

HeavyRatio	Iter.	p	grainSize	TP, мс	WS, мс	Лучший
0,01	5000	8	1024	30,045	29,867	WS
0,01	10000	8	64	58,429	57,607	WS
0,10	5000	8	64	289,917	280,242	WS
0,10	10000	8	64	564,045	576,804	TP
0,25	5000	8	64	710,027	706,181	WS
0,25	10000	8	64	1395,402	1485,887	TP

На рис. 3.7 видно, что с ростом доли тяжёлых агентов увеличивается не только среднее время, но и разброс между лучшими и худшими кадрами. Это ожидаемо: чем больше дорогих агентов попадает в задачи, тем выше вероятность неравномерного распределения работы между потоками. Для game loop это означает, что планировщик должен оцениваться не только по AvgFrameMs, но и по худшим кадрам.

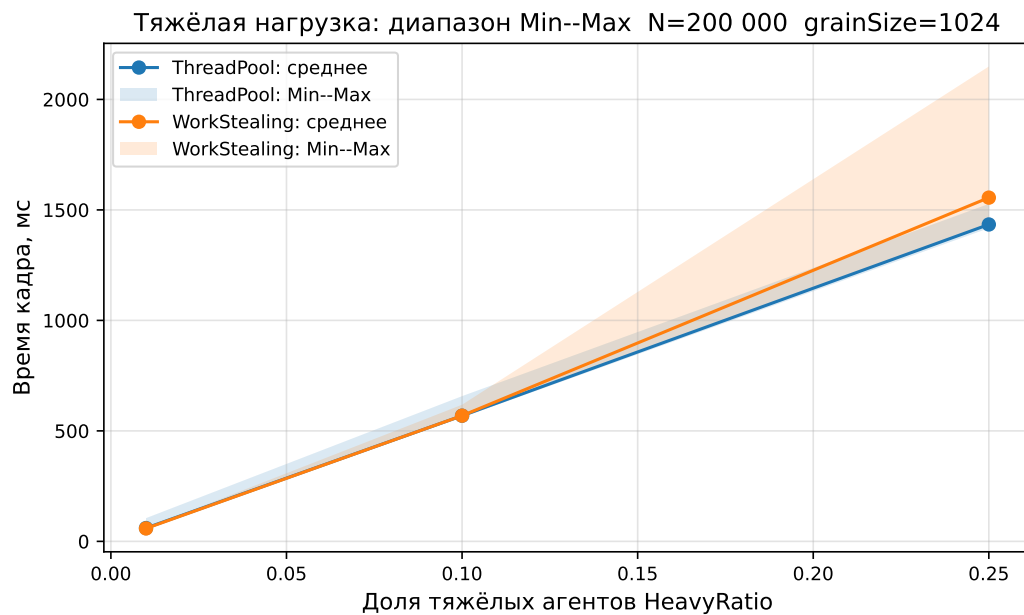


Рис. 3.7: Минимальное, среднее и максимальное время кадра в тяжёлой серии.

Дополнительная нормированная метрика показана на рис. 3.8. Она отражает относительный размах между минимальным и максимальным временем кадра. Такая нормировка удобна, потому что при росте HeavyRatio абсолютные времена увеличиваются на порядок, и прямое сравнение миллисекунд хуже показывает стабильность.

В ряде срезов WorkStealing уменьшает худший кадр, но этот эффект также не является гарантированным. При высокой доле heavy agents стоимость кражи и локальная структура распределения задач могут привести к

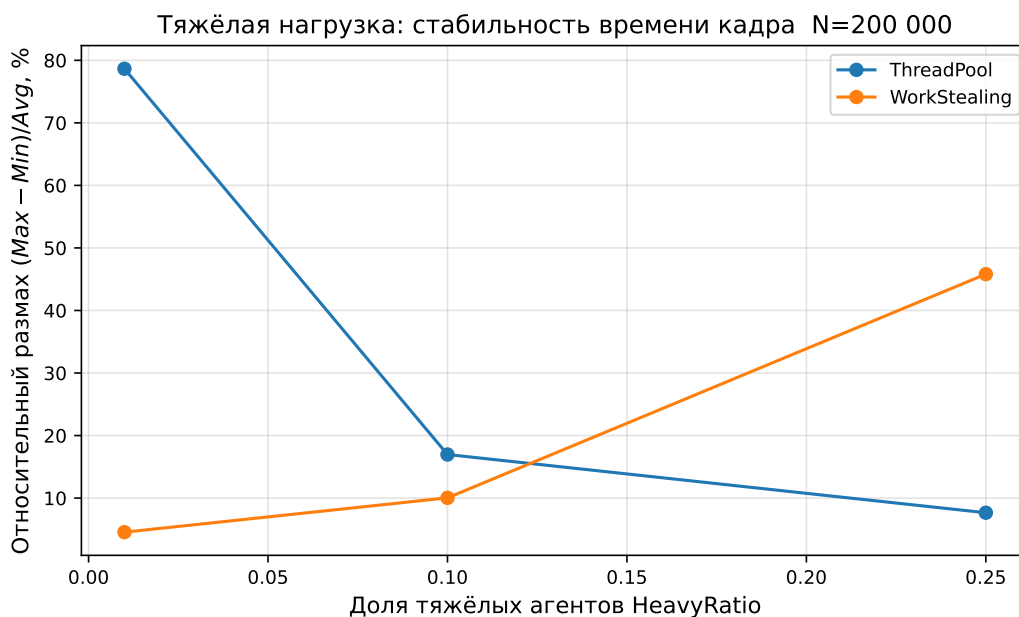


Рис. 3.8: Нормированный разброс времени кадра при разной доле тяжёлых агентов.

тому, что простой `ThreadPool` окажется стабильнее. Поэтому для практического применения нужно смотреть на распределение времени кадров, а не только на среднее значение.

3.7 Размер задачи при тяжёлой нагрузке

При тяжёлой нагрузке оптимальный `grainSize` может быть меньше, чем в `light`-сценарии. Причина в том, что каждая задача содержит больше полезной работы, а значит, дополнительные задачи не так сильно увеличивают относительный накладных расходов. При этом меньшие блоки улучшают балансировку тяжёлых агентов между потоками.

На рис. 3.9 показано влияние `grainSize` на тяжёлую серию. Для части конфигураций лучший результат достигается при `grainSize=64` или `256`, то есть при более мелком разбиении, чем в лёгкой нагрузке. Это подчёркивает, что универсального размера блока нет: параметр должен подбираться под конкретную стоимость агента и распределение тяжёлой работы.

3.8 Масштабирование и сравнение планировщиков

При тяжёлой нагрузке полезная работа велика, поэтому относительная доля `scheduling` накладных расходов уменьшается. На рис. 3.10 показан переход с 4 на 8 потоков. В рассмотренных срезах ускорение близко к двукратному: примерно 1,87–1,94 для `ThreadPool` и 1,81–1,95 для `WorkStealing`.

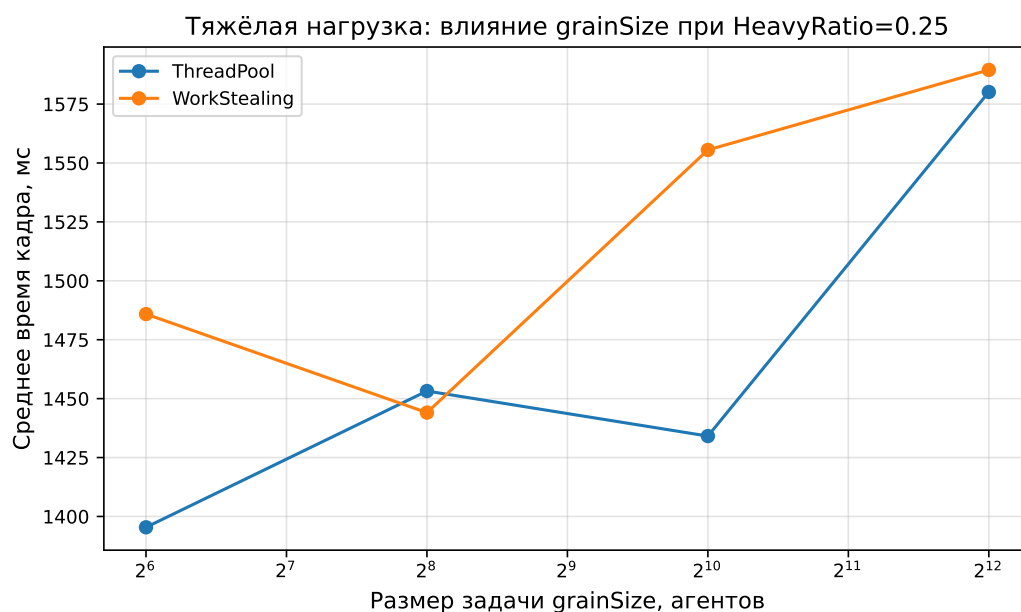


Рис. 3.9: Влияние grainSize на среднее время кадра при тяжёлой нагрузке.

Это подтверждает, что многопоточность наиболее полезна, когда задача содержит достаточно вычислений.

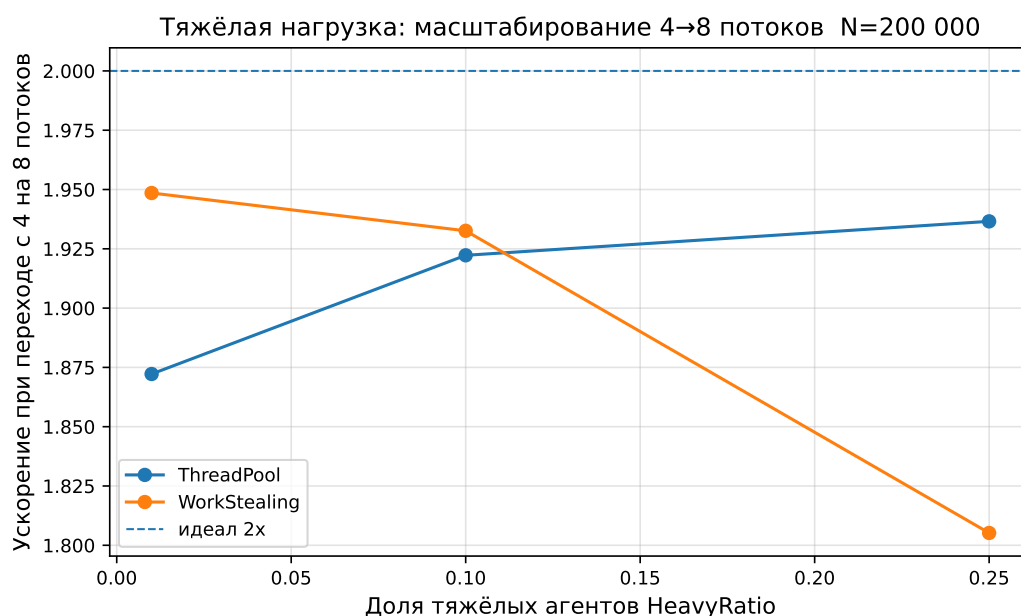


Рис. 3.10: Масштабирование тяжёлой нагрузки при переходе с 4 на 8 потоков.

Сравнение ThreadPool и WorkStealing показало отсутствие универсального победителя. В light-серии WorkStealing быстрее в 149 из 216 парных конфигураций, то есть в 69,0% пар, но при слишком малом grainSize обе реализации могут быть медленнее Sequential. В heavy-серии ThreadPool быстрее в 110 из 192 пар, а WorkStealing — в 82 из 192.

На рис. 3.11 приведено относительное сравнение двух параллельных планировщиков в тяжёлой серии. Значения около единицы означают близкую производительность. Видно, что при некоторых параметрах WorkStealing выигрывает, но при других уступает. Это особенно важно для инженерной интерпретации: наличие механизма stealing не гарантирует лучшего результата, если исходное разбиение уже достаточно сбалансировано или если steal-операции создают заметные расходы.

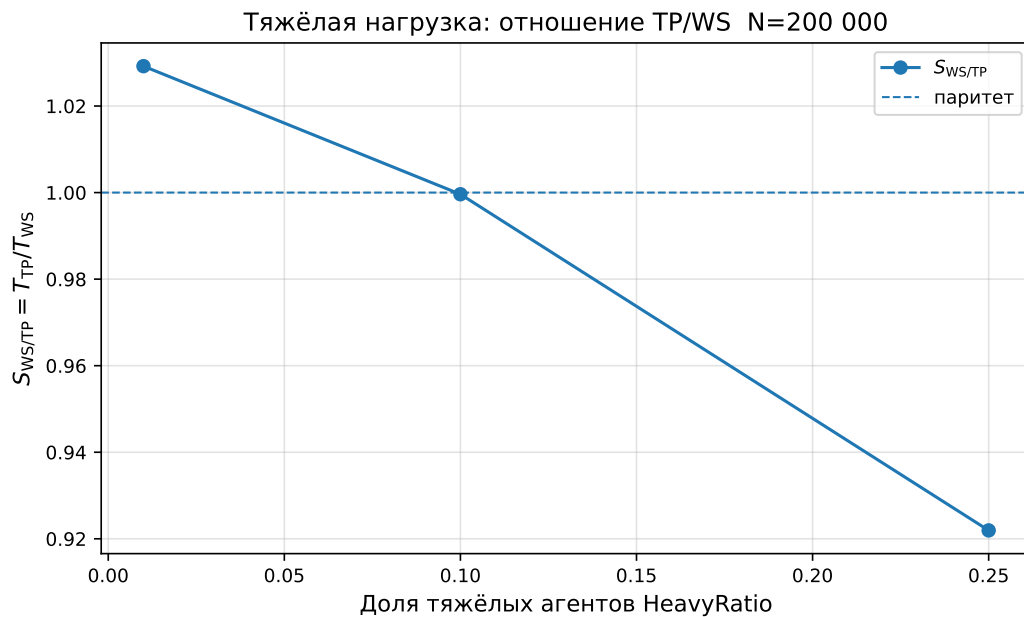


Рис. 3.11: Относительное сравнение WorkStealing и ThreadPool в тяжёлой серии.

Следовательно, work stealing следует рассматривать как инструмент для определённых типов дисбаланса, а не как безусловно более производительный планировщик. Если задачи однородны или стоимость stealing сравнима с выигрышем от балансировки, простой пул потоков может быть не хуже.

Полученные результаты также согласуются с законом Амдала. Если в кадре остаётся значительная последовательная часть или если планировщик добавляет существенные служебные расходы, увеличение числа потоков не даёт пропорционального ускорения. В light-серии это проявляется особенно явно: сама полезная работа мала, поэтому даже восемь потоков не помогают при слишком мелком grainSize. В heavy-серии полезная часть больше, и масштабирование между 4 и 8 потоками становится ближе к идеальному, хотя всё равно ограничивается балансировкой и накладных расходов.

С инженерной точки зрения это означает, что оптимизация должна начинаться не с выбора «самого сложного» планировщика, а с анализа структу-

ры работы. Если update агента занимает мало времени, выгоднее укрупнять блоки, улучшать layout данных и уменьшать синхронизацию. Если же агент дорогой и неоднородный, на первый план выходят балансировка и контроль худших кадров.

3.9 Исследование межагентных зависимостей на основе снимка состояния

Отдельная серия `interaction_snapshot` проверяет режим `SnapshotNeighbors`: агент читает снимок соседей в пределах радиуса R , а результаты применяются отдельно от параллельного прохода. CSV содержит 135 строк с последовательным базовый последовательный вариант и вариантами `ThreadPool/WorkStealing` на 4 и 8 потоках.

На рис. 3.12 показано усреднение по радиусам, размерам задач и числу потоков. Среднее ускорение растёт вместе с числом агентов: для `ThreadPool` 1,17, 2,13, 2,66 при $N = 10000, 50000, 100000$; для `WorkStealing` — 1,20, 2,10, 2,67. Значит, рост N увеличивает долю полезной работы относительно стоимости задач и барьеров.

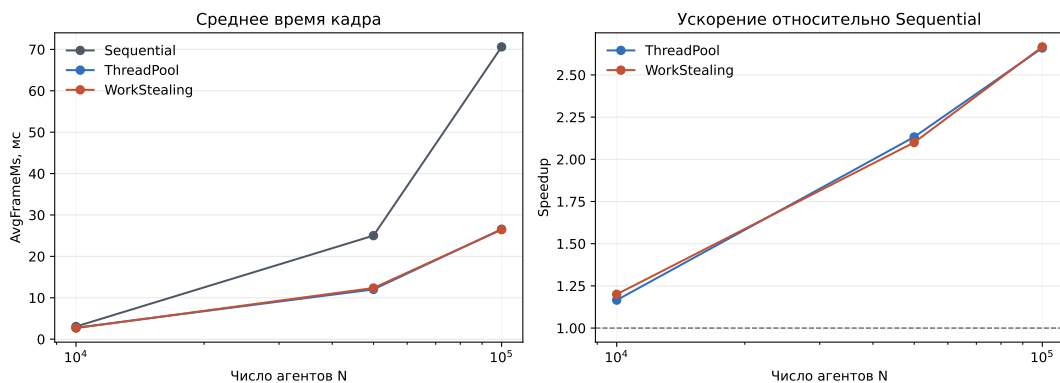


Рис. 3.12: Масштабирование режима `SnapshotNeighbors` по числу агентов.

Окупаемость зависит и от радиуса. При $N = 10000$, $R = 2$ параллельные варианты не превысили Sequential; при $R = 5$ ускорение появилось в 3 из 6 конфигураций `ThreadPool` и в 4 из 6 для `WorkStealing`; при $R = 10$ ускорились все варианты. Начиная с $N = 50000$, все проверенные параллельные конфигурации быстрее базовый последовательный вариант.

Влияние радиуса показано на рис. 3.13. Последовательное время растёт с 23,32 мс при $R = 2$ до 45,95 мс при $R = 10$. Среднее ускорение параллельных вариантов тоже растёт: для `ThreadPool` от 1,39 до 2,60, для `WorkStealing` от 1,37 до 2,60. Значит, межагентные чтения делают

параллелизм выгоднее, но чтение снимка усиливает нагрузку на память и не даёт масштабироваться пропорционально числу потоков.

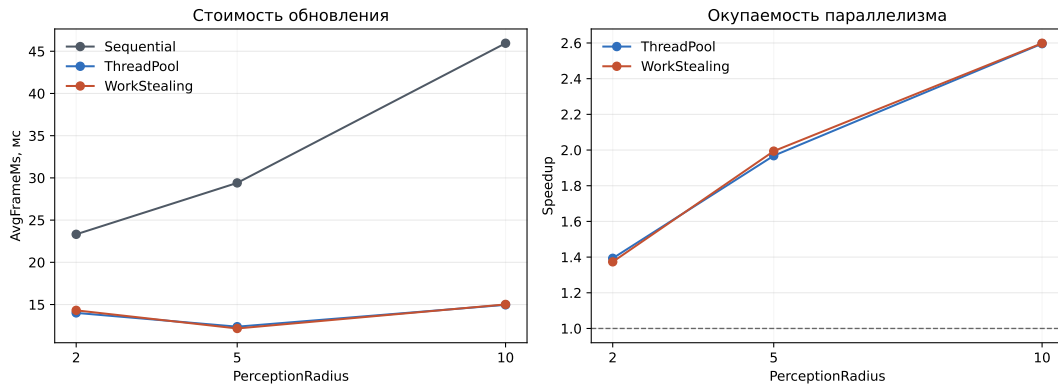


Рис. 3.13: Влияние радиуса межагентного восприятия на время кадра и ускорение.

Параметр `grainSize` менее чувствителен, чем в `light`-сценарии с микрозадачами. На рис. 3.14 среднее время при 64...1024 меняется умеренно. Единого оптимума нет: для `ThreadPool` размеры 64, 256 и 1024 выигрывают 7, 6 и 5 раз; для `WorkStealing` — 10, 7 и 1 раз. В отличие от `light`-серии, чтение соседей даёт достаточно работы даже малым блокам, поэтому мелкое разбиение чаще помогает балансировке.

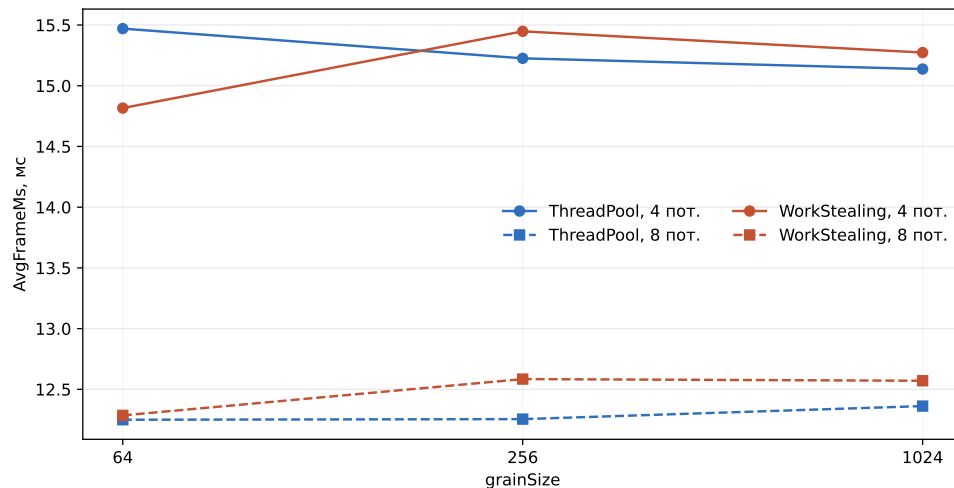


Рис. 3.14: Влияние `grainSize` на среднее время кадра в режиме `SnapshotNeighbors`.

Парное сравнение `ThreadPool` и `WorkStealing` выполнялось для одинаковых N , R , `grainSize` и числа потоков. На рис. 3.15 значение больше единицы означает преимущество `WorkStealing`. Из 54 пар `ThreadPool` быстрее в 26, `WorkStealing` — в 28; среднее отношение `TP/WS` равно 1,004, медиана — 1,001. Максимальные выигрыши сторон близки: 1,192 и 1,178 раза.

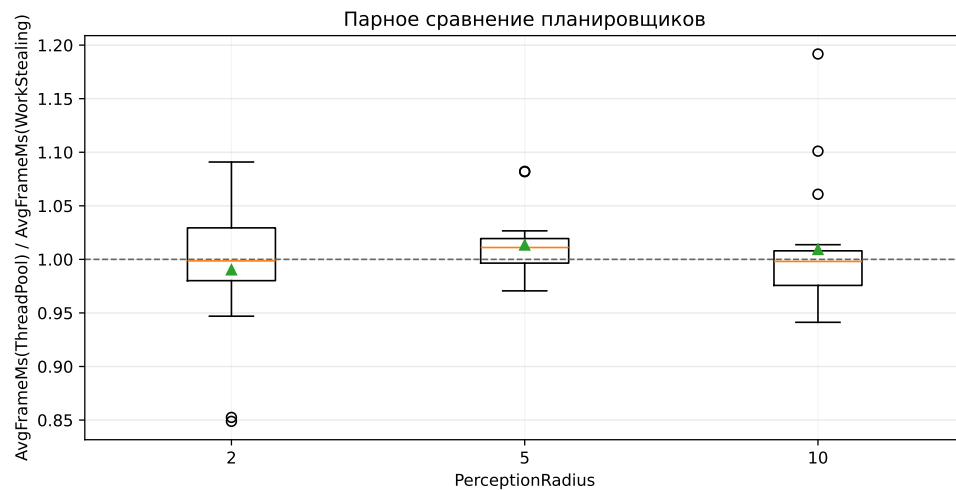


Рис. 3.15: Парное сравнение времени ThreadPool и WorkStealing.

Универсального победителя нет: при $R = 5$ WorkStealing выигрывает 12 пар против 6, а при $R = 2$ и $R = 10$ чаще выигрывает ThreadPool — по 10 пар против 8. Кража задач полезна при дисбалансе, но при ровных блоках или ограничении пропускной способностью памяти её накладные расходы могут перекрыть пользу.

Масштабирование с 4 до 8 потоков показано на рис. 3.16. Идеальный переход дал бы ускорение 2 раза, но средние значения равны только 1,20 для ThreadPool и 1,18 для WorkStealing; максимальные — 1,50 и 1,40. Ограничения создают последовательные участки, барьеры фаз и давление на память.

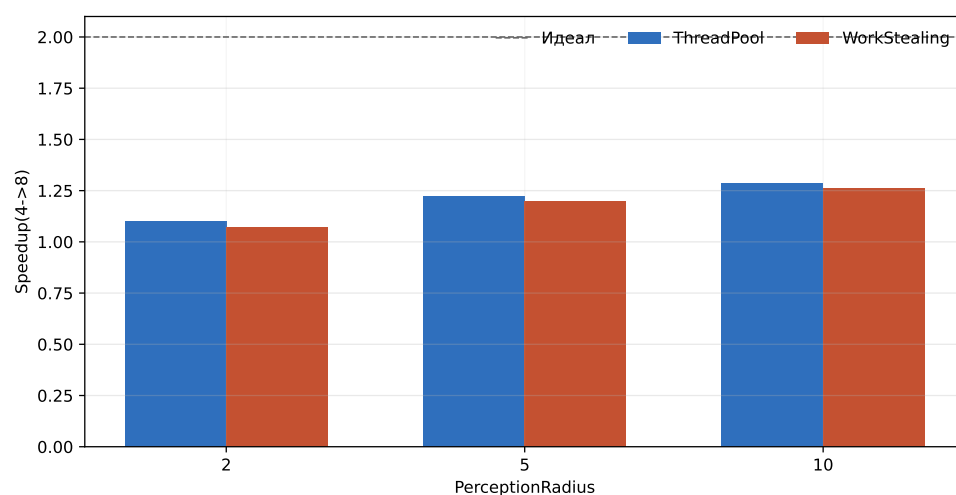


Рис. 3.16: Ускорение при переходе с 4 на 8 потоков в режиме SnapshotNeighbors.

Для игрового цикла важна стабильность худших кадров. На рис. 3.17 используется метрика $(\text{MaxFrameMs} - \text{MinFrameMs}) / \text{AvgFrameMs}$. Влия-

ние радиуса не монотонно: при $R = 5$ ThreadPool стабильнее, при $R = 10$ меньший средний разброс даёт WorkStealing. По худшему кадру преимущество минимально: 28 пар у ThreadPool против 26 у WorkStealing.

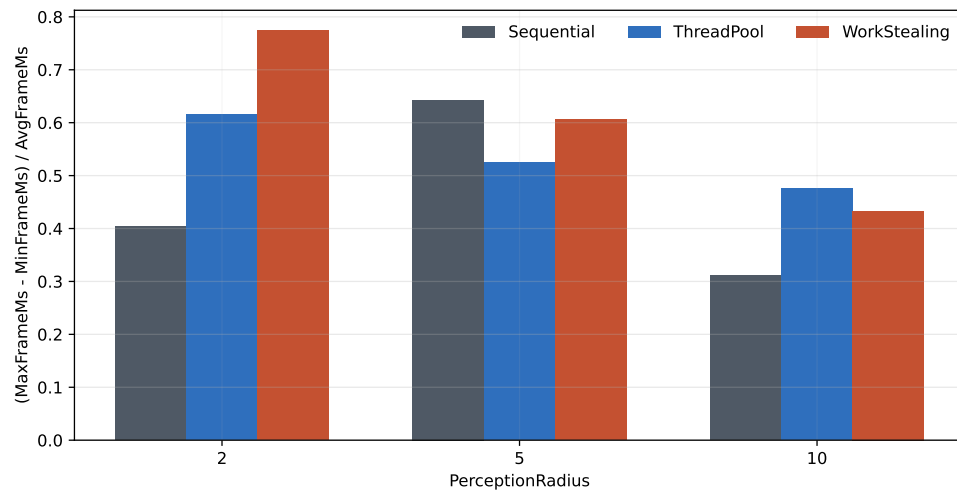


Рис. 3.17: Нормированный разброс времени кадра в режиме SnapshotNeighbors.

Размер задачи влияет и на стабильность. Для ThreadPool средний разброс при $g = 64, 256, 1024$ равен 0,631, 0,591 и 0,395; для WorkStealing — 0,464, 0,639 и 0,711. Значит, минимум среднего времени и минимум худших кадров могут требовать разных параметров.

Лучшие измеренные конфигурации приведены в табл. 3.5. Глобальный минимум в сетке соответствует $N = 10000$, $R = 10$, $grainSize=256$, 8 потокам и WorkStealing: 1,94 мс при ускорении 1,82. Это не универсальная рекомендация, а минимум среди измеренных строк; общий вывод важнее: подход на основе снимков состояния даёт полезную работу для параллелизма, но ограничивается памятью, фазовыми барьерами и балансировкой.

Таблица 3.5: Лучшие конфигурации режима SnapshotNeighbors

Срез	Знач.	N	R	g	p	Планировщик	Avg, мс	S
N	10000	10000	10	256	8	WorkStealing	1,94	1,82
N	50000	50000	5	64	8	WorkStealing	9,48	2,49
N	100000	100000	5	256	8	ThreadPool	20,18	3,09
R	2	10000	2	64	1	Sequential	3,17	1,00
R	5	10000	5	256	8	WorkStealing	2,23	1,09
R	10	10000	10	256	8	WorkStealing	1,94	1,82
Глобально	—	10000	10	256	8	WorkStealing	1,94	1,82

3.10 Практические рекомендации и ограничения

По результатам экспериментов можно выделить несколько рекомендаций. Во-первых, не следует распараллеливать слишком мелкие операции без

группировки в задачи достаточного размера. Во-вторых, `grainSize` необходимо подбирать экспериментально, потому что оптимум зависит от числа агентов, числа потоков, стоимости агента, радиуса межагентных чтений и реализации очередей. В-третьих, `ThreadPool` подходит для равномерной нагрузки и хорошо выбранного разбиения. В-четвёртых, `WorkStealing` целесообразен при выраженном дисбалансе, но его преимущество должно подтверждаться экспериментальным тестированием. Для межагентных зависимостей практичнее начинать с только для чтения `snapshot` и отдельной `commit`-фазы, а не с прямой конкурентной записи в состояние мира.

Таблица 3.6: Сводные рекомендации по выбору параметров

Ситуация	Практическое решение
Лёгкие однотипные агенты	Увеличивать <code>grainSize</code> до области, где накладных расходов перестает доминировать; сначала сравнивать с <code>Sequential</code> .
Тяжёлые агенты	Использовать несколько потоков; проверять масштабирование между 4 и 8 потоками и не выбирать слишком крупные блоки.
Неоднородная стоимость задач	Проверять <code>WorkStealing</code> в парном сравнении, но не считать его выигрыш гарантированным.
Требование стабильного кадра	Смотреть не только на <code>AvgFrameMs</code> , но и на <code>MaxFrameMs</code> и относительный разброс.
Межагентные зависимости	Читать состояние соседей из неизменяемого снимка, записывать решения в локальные буферы и применять конфликтующие изменения в отдельной <code>commit</code> -фазе.
Перенос в игровой движок	Повторять <code>benchmark</code> на реальной логике агентов, с фактическим <code>layout</code> -ом данных и зависимостями между подсистемами.

Работа имеет ограничения. Модель агента упрощена и не включает полноценную физику, анимацию, навигацию и сетевую синхронизацию; межагентные зависимости представлены только локальными чтениями снимка соседей, без сложного графа ресурсов и конфликтов. Реализация `WorkStealing` является исследовательской, а не промышленной системой управления задачами. Кроме того, результаты зависят от аппаратной платформы, компилятора и флагов оптимизации. Поэтому численные значения нельзя переносить в другие движки напрямую; основную ценность имеют выявленные закономерности.

3.11 Выводы по главе

Эксперименты подтвердили, что размер задачи является ключевым фактором производительности. В light-сценарии чрезмерно мелкий `grainSize` делает параллельное выполнение значительно медленнее последовательного. В heavy-сценарии масштабирование лучше, потому что полезная работа перекрывает служебные расходы. При этом `ThreadPool` и `WorkStealing` требуют парного сравнения: ни один из них не оказался универсально лучшим.

Режим `SnapshotNeighbors` показал, что межагентные зависимости на основе только для чтения снимка могут быть совместимы с параллельным update-ом. При малом числе агентов и малом радиусе восприятия накладные расходы всё ещё доминируют: для $N = 10000$, $R = 2$ параллельные варианты не окупались. При увеличении числа агентов или радиуса восприятия полезная работа растёт, и параллелизм становится выгоднее: среднее ускорение при переходе от $R = 2$ к $R = 10$ увеличилось примерно с 1,39 до 2,60.

Главные ограничения масштабирования в snapshot-серии связаны не только с планировщиком, но и с чтением памяти, барьерами между фазами и балансировкой задач. Переход с 4 на 8 потоков дал среднее ускорение 1,20 для `ThreadPool` и 1,18 для `WorkStealing`, что заметно ниже идеального значения 2. Парное сравнение планировщиков осталось почти симметричным: `ThreadPool` быстрее в 26 из 54 пар, `WorkStealing` — в 28 из 54. Поэтому выбор планировщика, `grainSize` и радиуса межагентных чтений должен проверяться измерениями на конкретной модели данных.

Заключение

В работе исследовано применение многопоточности к массовому обновлению игровых ИИ-агентов. Теоретический анализ показал, что такая нагрузка естественно выражается как параллельный проход по массиву данных, но корректность и эффективность требуют дисциплины доступа к состоянию мира, разделения параллельных и синхронизационных фаз, а также контроля накладных расходов.

В практической части разработана экспериментальная система задач с единым интерфейсом `IScheduler`. Реализованы три стратегии выполнения: `Sequential`, `ThreadPool` и `WorkStealing`. Для оценки производительности проведены серии бенчмарков с разным числом агентов, потоков, размером задач, долей тяжёлых агентов и радиусом межагентного восприятия.

Главный результат состоит в том, что многопоточность эффективна только при достаточном объёме полезной работы на задачу. При лёгкой нагрузке и `grainSize=1` планировщик `ThreadPool` оказался медленнее `Sequential` в 19,58 раза, а `WorkStealing` — в 10,34 раза. При увеличении `grainSize` накладные расходы уменьшаются, и параллельное выполнение начинает давать выигрыш.

Для тяжёлой нагрузки переход с 4 на 8 потоков даёт почти двукратное ускорение в показательных сценариях. Однако сравнение `ThreadPool` и `WorkStealing` не выявило универсального лидера: в `heavy`-серии `ThreadPool` быстрее в 110 из 192 парных конфигураций, а `WorkStealing` — в 82 из 192. Это подтверждает, что выбор планировщика должен основываться на измерениях.

Дополнительно исследован режим `SnapshotNeighbors`, в котором агенты читают неизменяемый снимок состояния соседей и применяют результаты через отдельную фазу. В этой серии параллелизм не окупился при $N = 10000$ и $R = 2$, но при $N = 50000$ и $N = 100000$ все проверенные параллельные конфигурации оказались быстрее последовательного базовый последовательный вариант. Рост радиуса восприятия увеличил среднее ускорение примерно с 1,39 при $R = 2$ до 2,60 при $R = 10$, что показывает: межагентные зависимости делают параллелизм полезнее, когда увеличивают объём работы на задачу.

В то же время подход на основе снимков состояния выявил ограничения масштабирования. Переход с 4 на 8 потоков дал среднее ускорение 1,20 для `ThreadPool` и 1,18 для `WorkStealing`, то есть заметно меньше идеального двукратного роста. Причины связаны с чтением памяти, синхронизацией фаз и балансировкой. В парном сравнении `snapshot`-серии `ThreadPool` выиграл 26 из 54 конфигураций, а `WorkStealing` — 28 из 54, поэтому универсального победителя нет и для режима межагентных зависимостей.

Практический вывод работы заключается в необходимости подход, основанный на экспериментальном тестировании к многопоточному ИИ. Нельзя заранее считать, что больше потоков, меньшие задачи или более сложный планировщик дадут лучший результат. Для зависимых агентов безопасной отправной точкой является только для чтения `snapshot` с локальными выходными буферами и `commit`-фазой; эффективность такой схемы определяется сочетанием архитектуры данных, `grainSize`, радиуса межагентных чтений, характера нагрузки и стоимости планирования.

Дальнейшее развитие работы связано с более реалистичной моделью агента: навигационными запросами, заявками на ресурсы, конфликтами между группами и сравнением `array-of-structures` с `structure-of-arrays layout`. Также полезно добавить полноценный граф зависимостей задач и более сложную `commit`-фазу, чтобы проверить найденные закономерности в архитектуре, близкой к `production job system`.

Литература

- [1] Unity Technologies. *Job system overview*. Unity Manual. URL: <https://docs.unity3d.com/6000.2/Documentation/Manual/job-system-overview.html>
- [2] Unity Technologies. *ParallelFor jobs*. Unity Manual. URL: <https://docs.unity3d.com/2020.1/Documentation/Manual/JobSystemParallelForJobs.html>
- [3] Unity Technologies. *IJobParallelFor*. Unity Scripting API. URL: <https://docs.unity3d.com/ScriptReference/Unity.Jobs.IJobParallelFor.html>
- [4] Unity Technologies. *The safety system in the C# Job System*. Unity Manual. URL: <https://docs.unity3d.com/2020.1/Documentation/Manual/JobSystemSafetySystem.html>
- [5] Epic Games. *Tasks Systems in Unreal Engine*. Unreal Engine Documentation. URL: <https://dev.epicgames.com/documentation/unreal-engine/tasks-systems-in-unreal-engine>
- [6] Epic Games. *Task Graph Insights in Unreal Engine 5*. Unreal Engine Documentation. URL: <https://dev.epicgames.com/documentation/unreal-engine/task-graph-insights-in-unreal-engine-5>
- [7] Gyrling, C. *Parallelizing the Naughty Dog Engine Using Fibers*. Game Developers Conference, 2015. URL: <https://www.gdcvault.com/play/1022186/parallelizing-the-naughty-dog-engine>
- [8] Game Developers Conference. *Task-based Multithreading: How to Program for 100 cores*. GDC Vault. URL: <https://gdcvault.com/play/1012321/Task-based-Multithreading-How-to>
- [9] Intel. *How Task Scheduler Works*. Intel oneAPI Threading Building Blocks Documentation. URL: <https://www.intel.com/content/www/>

us/en/docs/onetbb/developer-guide-api-reference/
2021-6/how-task-scheduler-works.html

- [10] Intel. *parallel_for*. Intel oneAPI Threading Building Blocks Documentation. URL: <https://www.intel.com/content/www/us/en/docs/onetbb/developer-guide-api-reference/2021-9/parallel-for.html>
- [11] Microsoft. *Thread Pools*. Win32 Apps Documentation. URL: <https://learn.microsoft.com/en-us/windows/win32/procthread/пулпотокoвs>
- [12] Amdahl, G. M. *Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities*. AFIPS Conference Proceedings, 30, 483–485, 1967. URL: <https://www3.cs.stonybrook.edu/~rezaul/Spring-2012/CSE613/reading/Amdahl-1967.pdf>